

SDD Competitive Analysis

Ed Barlow www.webcloudstudio.com August 5, 2026



SDD Competitive Analysis

Key	Product	What it is	License	Runs on
DD	Drydock	Specification builder based on Agile and Test Driven Development	MIT	Your LLM subscription
SK	GitHub Spec Kit	Prompt templates turning an idea into spec, plan, and tasks	MIT	Your LLM subscription
KI	Kiro (AWS)	IDE with a spec-first mode and agent hooks	Proprietary	Metered inference
BM	BMAD-METHOD	Multi-agent role framework for elicitation and story delivery	MIT	Your LLM subscription
CM	Plain CLAUDE.md	One rules file and an agent	Free	Your LLM subscription

Spec Kit, Kiro, and BMAD author specifications.

Drydock builds from them and does not author.

Summary

Section	DD	SK	KI	BM	CM	Winner
1 Specification handling	3.8	2.0	2.6	3.0	1.2	Drydock
2 Planning	5.0	1.4	2.0	2.6	0.2	Drydock
3 Build execution	4.2	1.0	1.7	2.5	1.2	Drydock
4 Verification	4.8	0.3	1.2	1.2	0.7	Drydock
5 Change management	4.8	0.6	1.2	1.2	0.2	Drydock
6 Context and cost	4.8	1.3	0.5	2.3	1.8	Drydock
7 Governance	4.8	1.5	2.0	1.8	2.3	Drydock
8 Review surface	4.8	0.5	2.5	1.8	0.3	Drydock
9 Adoption cost	1.8	3.7	2.7	3.3	5.0	CLAUDE.md
Overall	4.2	1.4	1.8	2.2	1.5	Drydock

Drydock wins every section except adoption cost, where it comes last.

Under a week of work, a rules file beats it.

1 Specification Handling

#	Capability	DD	SK	KI	BM	CM	Winner
1.1	Creates a specification from a blank page	● 0	● 4	● 4	● 5	● 2	BMAD
1.2	Ingests existing source material as the input of record	● 5	● 1	● 2	● 3	● 2	Drydock
1.3	Typed specification files with prescribed roles	● 5	● 3	● 3	● 3	● 1	Drydock
1.4	Specification quality audit against the raw sources	● 4	● 0	● 1	● 2	● 0	Drydock
1.5	Specification stays authoritative after code exists	● 5	● 2	● 3	● 2	● 1	Drydock

Drydock scores zero on authoring. It is an explicit non-goal.

2 Planning

#	Capability	DD	SK	KI	BM	CM	Winner
2.1	Dependency graph with a computed runnable frontier	● 5	● 1	● 1	● 2	● 0	Drydock
2.2	Stories with stable IDs, states, and dependency edges	● 5	● 3	● 4	● 4	● 0	Drydock
2.3	Effort estimated as token cost	● 5	● 0	● 0	● 2	● 0	Drydock
2.4	Blocker gate that halts planning until the human answers	● 5	● 1	● 2	● 3	● 0	Drydock
2.5	Human-editable plan: reorder, regroup, split	● 5	● 2	● 3	● 2	● 1	Drydock

The others emit a task list. `MANIFEST.md` is a graph database with build state and provenance.

3 Build Execution

#	Capability	DD	SK	KI	BM	CM	Winner
3.1	Executes the plan block by block to working software	● 5	● 3	● 4	● 4	● 3	Drydock
3.2	Per-step context stacking — only the specs that step needs	● 5	● 2	● 2	● 4	● 1	Drydock
3.3	Automatic repair loop with a convergence rule	● 5	● 0	● 1	● 2	● 1	Drydock
3.4	Model escalation on the final repair attempt	● 5	● 0	● 0	● 1	● 0	Drydock
3.5	Parallel or worktree-isolated execution	● 0	● 0	● 1	● 2	● 1	BMAD
3.6	Resume in place with partial work and failing checks intact	● 5	● 1	● 2	● 2	● 1	Drydock

Repair continues only while the passing set grows without regression. That is a convergence rule, not a retry counter.

Drydock owns the graph that makes parallelism safe and builds serially anyway.

4 Verification

#	Capability	DD	SK	KI	BM	CM	Winner
4.1	TDD gate — criteria exist and fail before implementation	 4	 1	 2	 3	 2	Drydock
4.2	Acceptance verified by executing it, no LLM in the loop	 5	 0	 1	 1	 0	Drydock
4.3	Vacuous or tautological proofs demoted to UNVERIFIED	 5	 0	 0	 0	 0	Drydock
4.4	EARS project acceptance criteria with stable IDs	 5	 0	 3	 0	 0	Drydock
4.5	Guardrails as absolute prohibitions behind a hard gate	 5	 1	 0	 1	 2	Drydock
4.6	Release verdict with enumerated blockers	 5	 0	 1	 2	 0	Drydock

The widest margin here. Everyone else lets the agent that wrote the code decide whether the code is correct.

4.1 is not a 5: the proof is not hash-locked before implementation, so an agent can still shape its own target.

5 Change Management

#	Capability	DD	SK	KI	BM	CM	Winner
5.1	Drift detection by content hash and commit provenance	 5	 0	 1	 0	 0	Drydock
5.2	Sealed foundational specs requiring a change ticket	 5	 0	 0	 1	 0	Drydock
5.3	Incremental replan scoped to impacted work only	 5	 1	 2	 2	 0	Drydock
5.4	Change the spec, then rebuild — the normal path	 5	 2	 3	 3	 1	Drydock
5.5	Regression reopens a verified sibling story	 4	 0	 0	 0	 0	Drydock

The others are strongest on the first pass and weakest on the second.

6 Context and Cost

#	Capability	DD	SK	KI	BM	CM	Winner
6.1	Compaction of rules and specs to a callable surface	 5	 0	 0	 3	 1	Drydock
6.2	Full-versus-compact selection driven by the build graph	 5	 0	 0	 0	 0	Drydock

#	Capability	DD	SK	KI	BM	CM	Winner
6.3	No API-key path; subscription CLI enforced	● 5	● 3	● 0	● 3	● 5	Drydock / CLAUDE.md
6.4	Large project delivered with a cheaper model	● 4	● 2	● 2	● 3	● 1	Drydock

Context is treated as a budgeted resource. Nobody else prices it.

7 Governance

#	Capability	DD	SK	KI	BM	CM	Winner
7.1	Injected business rules, stack rules, and branding	● 5	● 2	● 3	● 2	● 4	Drydock
7.2	Persistent intent injected at every stage	● 5	● 3	● 3	● 3	● 4	Drydock
7.3	Reviewable build evidence written per step	● 5	● 1	● 2	● 2	● 0	Drydock
7.4	Dependency legitimacy guard on generated packages	● 4	● 0	● 0	● 0	● 1	Drydock

A CLAUDE.md gets most of 7.1 and 7.2 for no setup cost. Rigging adds versioning, compaction, and auditable evidence.

8 Review Surface

#	Capability	DD	SK	KI	BM	CM	Winner
8.1	Review console for plan, evidence, and decisions	● 5	● 0	● 4	● 1	● 0	Drydock
8.2	Questionnaires whose answers persist into later commands	● 5	● 1	● 2	● 3	● 0	Drydock
8.3	Board view of plan state	● 4	● 0	● 3	● 1	● 0	Drydock
8.4	Generated documentation with publishable rendering	● 5	● 1	● 1	● 2	● 1	Drydock

Kiro reviews the specification. The QuarterDeck reviews per-step build evidence.

9 Adoption Cost

#	Capability	DD	SK	KI	BM	CM	Winner
9.1	Time from zero to first working code	● 1	● 3	● 4	● 2	● 5	CLAUDE.md
9.2	Right-sized for a one-file fix	● 1	● 2	● 3	● 3	● 5	CLAUDE.md
9.3	Zero install, no new tooling	● 1	● 3	● 1	● 2	● 5	CLAUDE.md
9.4	Open source, no license cost	● 5	● 5	● 1	● 5	● 5	All but Kiro
9.5	Language and stack coverage out of the box	● 2	● 4	● 4	● 4	● 5	CLAUDE.md

#	Capability	DD	SK	KI	BM	CM	Winner
9.6	Community and third-party ecosystem	1	5	3	4	5	Spec Kit / CLAUDE.md

Drydock ships Python Rigging only and has no ecosystem.

`import` → `analyze` → `plan` → `build` is a long path to first running software.

Which To Use

Situation	Use
One-file fix or exploratory hacking	Plain CLAUDE.md
Nothing written down yet	BMAD-METHOD
Free, agent-neutral team convention	GitHub Spec Kit
IDE-centric team, spec-first with editor hooks	Kiro
Large system, spec already written, months of work	Drydock
Delivery that must be audited or defended	Drydock
Long-lived product whose spec keeps changing	Drydock
Maximum agent throughput on independent work	None of these

The natural pairing is BMAD or Kiro upstream, Drydock downstream.

The Wider Field

Eleven systems on the same scale. Maturity follows Böckeler's taxonomy: **spec-first** (the spec guides the first build, then drifts), **spec-anchored** (spec and code co-evolve under tests), **spec-as-source** (humans edit only specs).

Key	Product	License	Maturity
DD	Drydock	MIT	Spec-anchored
SK	GitHub Spec Kit	MIT	Spec-first
KI	Kiro (AWS)	Proprietary	Spec-first
TS	Tessl + intent-integrity-kit	Proprietary	Spec-as-source
OS	OpenSpec	MIT	Spec-anchored
BM	BMAD-METHOD	MIT	Spec-first
KT	Spec Kitty	Open source	Spec-anchored
WA	Walden	Open source	Spec-anchored
SP	Superpowers	Open source	Spec-first
GS	GSD (Get Shit Done)	Open source	Spec-anchored
TR	Traycer	Proprietary	Spec-first

Drydock's Ground

#	Feature	DD	SK	KI	TS	OS	BM	KT	WA	SP	GS	TR
A1	Typed specification files with prescribed roles	5	3	3	4	3	3	3	3	1	3	2
A2	Dependency-graph plan with runnable frontier	5	1	1	1	1	2	2	2	2	4	2
A3	Context-size-aware prompt stacking	5	1	1	2	2	4	2	1	3	4	3
A4	Story points as token cost	5	0	0	0	0	2	0	0	0	1	0
A5	EARS acceptance criteria, grammar-validated	5	0	2	0	0	0	0	5	0	0	0
A6	Guardrails as absolute prohibitions with a hard gate	5	2	0	2	0	0	1	3	0	2	0
A7	Evidence bound to Git HEAD and content hashes	5	0	0	3	0	0	1	4	1	2	0
A8	Discount for model-judged verification	5	0	0	2	0	0	0	2	0	0	0
A9	Drift detection via applied-spec hashing	4	0	0	4	2	0	1	5	0	2	1
A10	Sealed foundational specs requiring a ticket	5	1	0	2	3	0	1	3	0	1	0
A11	Review console for plan, evidence, and decisions	4	1	4	2	3	2	4	2	1	1	5
A12	Persistent intent injected into every prompt	5	4	3	3	3	3	3	4	2	3	2
A13	Enterprise branding and stack rules injection	5	2	3	1	1	2	1	2	1	1	1
A14	Spec compaction to cut context cost	5	0	0	2	1	3	0	0	1	2	1
A15	Blocker loop that halts planning	5	1	2	1	2	3	2	3	3	3	2
A16	Brownfield reverse-engineering into specs	3	1	2	2	4	3	2	1	1	2	3
A17	Documentation generation and publishable rendering	5	1	1	2	1	2	1	1	0	1	1
A18	Subscription-CLI-only execution	5	3	0	0	3	3	3	4	4	3	0
A19	Provider and IDE neutrality	4	5	0	2	4	4	5	5	2	2	3

Contested: A5 and A9 to Walden, A11 to Traycer, A19 to the portable open-source tools.

The Field's Ground

#	Feature	DD	SK	KI	TS	OS	BM	KT	WA	SP	GS	TR
B1	Worktree isolation for parallel agents	0	0	0	0	1	2	5	0	5	4	3
B2	Parallel or wave execution	0	0	0	0	1	3	5	0	4	5	3
B3	Enforced TDD RED→GREEN cycle	4	1	2	3	1	3	2	3	5	3	1
B4	Test artifacts hash-locked before implementation	0	0	0	5	0	0	0	2	1	0	0
B5	Test-quality analysis	5	0	0	5	0	0	0	1	2	2	0
B6	Delta change format plus archive	2	1	1	2	5	1	2	2	0	2	1
B7	CI release gate with exit-code enforcement	1	1	0	2	2	0	2	5	0	2	0
B8	Semantic diff and downstream impact warning	1	0	0	5	3	0	1	4	0	2	1
B9	Retrospectives and lessons capture	1	0	0	1	1	2	5	4	2	2	1
B10	Adaptive rigor — workflow right-sized	1	1	1	2	4	5	2	1	3	4	4
B11	Explore or spike phase before planning	2	2	1	3	5	4	2	2	5	5	4
B12	Cross-feature conflict analysis	1	1	1	2	2	3	1	2	1	2	2
B13	Decision-coverage gate	1	0	0	3	1	1	1	2	1	5	1
B14	Hallucinated or malicious package detection	4	0	0	0	0	0	0	0	0	5	0
B15	Dynamic model routing and cost tiering	1	0	0	1	0	2	1	0	2	5	3
B16	Gherkin executable behavioral specs	0	1	2	4	3	2	2	2	1	1	1
B17	Spec bloat and reviewability control	2	1	1	2	3	1	2	3	2	2	2
B18	Cross-repository spec sharing	0	0	0	4	3	1	0	0	0	0	0
B19	Fix the spec first, then regenerate code	2	1	1	5	3	1	2	3	1	2	1
B20	Multi-agent role specialization	1	2	2	2	1	5	4	1	4	4	3

Since the 2026-07-17 edition: B3 rose 1→4 on RED→GREEN enforcement, B5 rose 0→5 on proof-integrity demotion, B14 rose 0→4 on dependency legitimacy guardrails.

The nearest competitor is Walden, not Spec Kit or Kiro. It reached EARS validation, staleness chains, and proof-based completion independently.

Where Drydock Must Improve

1. **Right-sizing.** No lightweight path. A one-line change pays the full pipeline.
2. **Parallel execution.** Scores zero while owning the dependency graph that makes it safe.
3. **Authoring.** A deliberate non-goal, but it means Drydock cannot be a team's only tool.
4. **Stack coverage.** Python Rigging only. Every other team writes governance before evaluating.
5. **Proof locking.** Criteria are gated but not hash-locked before implementation.
6. **CI gate.** The release verdict is not packaged as an exit code a pipeline can enforce.

Legend

Mark	Score	Meaning
	5	Best in field; deterministic and enforced
	4	Strong first-class capability
	3	Present; advisory rather than enforced
	2	Partial or manual
	1	Nominal or documentation-only
	0	Absent

Method

Analysis date 2026-08-05, superseding the 2026-07-17 edition. Drydock is scored against `docs/Drydock_Specification.md` as implemented. Every other product is scored from published primary documentation; none was executed. Scores are engineering judgment, not measurements.

Sections 1–8 are drawn from the problem Drydock solves, which biases the summary in its favor. Section 9 and the improvement list are the counterweight.

Sources

- Böckeler, *Understanding Spec-Driven Development: Kiro, spec-kit, and Tessel* — <https://martinfowler.com/articles/exploring-gen-ai/sdd-3-tools.html>
- *Spec-Driven Development: From Code to Contract* — <https://arxiv.org/html/2602.00180v1>
- cameronjsj/spec-compare — <https://github.com/cameronjsj/spec-compare>
- *Skepticism Toward Specification-Driven Development* — <https://zenn.dev/cbmrham/articles/202601-spec-driven-development-skepticism>
- GitHub Spec Kit — <https://github.com/github/spec-kit>
- Kiro — <https://kiro.dev/docs/>
- BMAD-METHOD — <https://docs.bmad-method.org/>

- OpenSpec — <https://github.com/Fission-AI/OpenSpec>
- Spec Kitty — <https://github.com/Priivacy-ai/spec-kitty>
- Walden — <https://andrearaponi.github.io/walden/>
- Superpowers — <https://github.com/obra/superpowers>
- GSD — <https://github.com/gsd-build/get-shit-done>
- Traycer — <https://docs.traycer.ai/quickstart>
- Tessl intent-integrity-kit — <https://tessl.io/registry/tessl-labs/intent-integrity-kit/2.7.5>